FleetTrack — Plan de pruebas

Curso 048 Aseguramiento de la Calidad del Software. Responsable de consolidación y aprobación: Alejandro. Aportes: Manuel (web, Docker, Ansible, Cypress, ZAP, accesibilidad, usabilidad) y Andrea (Android, consumidor, Maestro, k6, Sonar).

1. Introducción y objetivo

Demostrar, con evidencia reproducible, que FleetTrack cumple los requisitos funcionales y de calidad (seguridad, rendimiento, accesibilidad, recuperación ante fallos) y emitir una decisión GO / NO-GO fundamentada. Cada caso del archivo `casos-prueba.xlsx` enlaza requisito, prueba, resultado, evidencia y defecto; cada reporte indica fecha, commit y ambiente (`reports/metadata.json`).

2. Alcance

Dentro	Fuera
Dashboard web, API REST, consumidor RabbitMQ, app Android, mensajería, caché, base de datos, contenedores	Alta disponibilidad, balanceo, iOS, pagos, mapas
Seguridad de aplicación, dependencias, contenedores, rendimiento de `POST /api/tracking`, accesibilidad WCAG 2 A/AA	Pruebas de penetración manuales, cumplimiento legal de datos de ubicación
Terraform y Ansible (validación de sintaxis; ejecución real sujeta a Azure)	Costos de nube

3. Arquitectura bajo prueba

Resumen: dashboard React y app Android → API .NET → PostgreSQL (con outbox) y Redis; el outbox publica en RabbitMQ y un consumidor idempotente actualiza la caché. Detalle en `arquitectura.pdf`.

4. Riesgos que guían la estrategia

Detalle, probabilidad, impacto y mitigación en `matriz-riesgos.xlsx`. Los de mayor prioridad: duplicación o pérdida de operaciones (R01, R02), caída de Redis o del broker (R03, R04), pérdida de datos en el móvil sin conexión (R05), acceso a datos ajenos (R06) y vulnerabilidades de dependencias (R08). Cada uno tiene al menos una prueba asociada en la matriz de trazabilidad.

5. Estrategia

Pirámide de pruebas con automatización total en Jenkins (`Jenkinsfile`):

Nivel	Herramienta	Cuándo
Unitarias	xUnit	Cada build
Integración y API	xUnit contra el stack real en Docker; colección Insomnia	Cada build
Funcionales y regresión web	Cypress	Cada build
Móvil	Maestro sobre emulador (agente Android)	Cada build
Recuperación	`tests/recovery/run-all.sh` (26 verificaciones)	Cada build
Seguridad	Semgrep (SAST), SonarQube, ZAP, npm audit, NuGet audit, Dependency-Check	Cada build
Rendimiento	k6 (corto en cada build; escala completa en el candidato a release)	Candidato a release
Usabilidad	5 participantes con tareas definidas	Antes del GO

Orden del pipeline: checkout → dependencias → lint → unitarias → SonarQube → npm audit → Dependency-Check → SAST → build de imágenes → despliegue QA → API → Cypress → recuperación → Maestro → ZAP → rendimiento → Quality Gate y criterios de salida → aprobación de Alejandro → despliegue final. Ningún paso ignora errores.

6. Ambientes y datos

Ambiente	Descripción
QA local	`docker compose up -d --build`; PostgreSQL 16 (puerto host 5433), Redis 7, RabbitMQ 3, API 8080, consumidor, web 3000
Android	Emulador Pixel 6, Android 15 (API 35, x86_64), API en `10.0.2.2:8080`
CI	Jenkins en equipo propio: agente `qa` y agente `android`

Datos: cuentas de prueba del seed (supervisor y conductor de prueba), clientes y vehículos creados por las propias pruebas con nombres únicos. Ningún dato real. Secretos en `.env` o credenciales de Jenkins, nunca en Git.

7. Pruebas unitarias

6 pruebas xUnit: creación del token JWT con los claims de usuario, correo y rol, y error si falta el secreto (`JwtTokenServiceTests`); y la regla de orden del consumidor `EventProcessor.IsNewer` en cuatro casos: sin caché, evento posterior, evento atrasado y mismo instante (`EventProcessorTests`). Cobertura medida con `dotnet-coverage` (`tests/coverage/run-coverage.sh`) y enviada a SonarQube.

8. Pruebas de integración

22 pruebas xUnit contra la API y PostgreSQL reales (`backend/tests/FleetTrack.Api.IntegrationTests`): login inválido (401), `drivers/me` sin token (401), con token de supervisor (403) y de conductor (200), conductor que no puede operar rutas ni entregas ajenas, creación de vehículo sin rol de supervisor (403), asignación de ruta y consulta por el conductor, completar dos veces la misma entrega (idempotente), diez `complete` concurrentes, completar una entrega fallida (409), coordenadas fuera de rango (400), reenvío del mismo `eventId` de tracking llegada de una ubicación al broker y al `EventLogs` del consumidor, y la administración de clientes, vehículos y conductores (creación, listado, placa y correo duplicados con 409, contraseña débil y capacidad omitida con 400, y listados solo para supervisor). La cobertura que generan se mide ejecutando la API bajo `dotnet-coverage`.

9. Pruebas de API

Las anteriores cubren los contratos principales (códigos 200/400/401/403/409 y validación de campos obligatorios con `JsonRequired`); la evidencia fotográfica idempotente se verifica en el experimento E7 de recuperación. La colección `tests/insomnia/FleetTrack.insomnia.json` permite recorrer los endpoints a mano; no se automatiza con `inso` porque cada petición necesita un token. ZAP API analiza `/health` (no se publica OpenAPI; ver riesgos residuales).

10. Pruebas funcionales y smoke

Cypress: `smoke` (arranque y login), `flujo-critico-asignar-ruta` (crear cliente, vehículo y conductor, y asignar una ruta), `negativo` (credenciales inválidas, placa duplicada, formularios vacíos). Smoke posterior a cada despliegue: `/health` de la API y la página principal del dashboard.

11. Regresión

Suite completa tras cada cambio: 6 unitarias + 22 integración + 26 de recuperación + 15 Cypress + 8 Maestro = **77 verificaciones automatizadas**. La regresión de la semana 7 ya detectó un defecto de aislamiento entre suites (la caché de Redis quedaba con una posición más nueva), corregido en el propio experimento E2.

12. Pruebas móviles

Ocho flujos Maestro sobre el emulador: login, consulta de ruta, inicio de entrega, envío de ubicación, evidencia fotográfica, desconexión, sincronización y finalización. `tests/maestro/run-all.sh` crea una ruta por flujo y apaga/reactiva la red del emulador en los flujos 06 y 07. Validación manual adicional: entrega fallida con motivo y mensaje de operación guardada sin conexión.

13. Seguridad

SAST con Semgrep (sustituto de Fortify, ver riesgos residuales), SonarQube (bugs, vulnerabilidades, hotspots), ZAP baseline y API, `npm audit`, `dotnet list package --vulnerable`, Dependency-Check (requiere clave de NVD) y análisis de autorización (pruebas de integración de propiedad de recursos). Contenedores sin privilegios.

14. Rendimiento

k6 sobre `POST /api/tracking`. Load: 100 conductores, una ubicación cada 5 s durante 10 min, umbral p95 < 500 ms y errores < 1 %. Stress: aumento de carga hasta 300 usuarios virtuales. Spike: salto de 100 a 300. Se registra saturación, colas y recuperación, sin pérdida de operaciones aceptadas ni duplicados. En cada build corre una versión corta.

15. Usabilidad y accesibilidad

Accesibilidad: axe-core (WCAG 2 A/AA) sobre cuatro pantallas dentro de Cypress; criterio de salida: cero hallazgos críticos o serios. Usabilidad: cinco participantes, siete tareas, éxito ≥ 80 % de los intentos y registro de dificultades (`docs/usabilidad`).

16. Criterios de entrada y salida

Entrada	Salida (todos obligatorios para GO)
Rama integrada en `main` con checks verdes; ambiente QA arriba y sano; datos de prueba sembrados	Cero pruebas críticas fallidas; Quality Gate de SonarQube aprobado sin cambiar umbrales; sin vulnerabilidades críticas no aceptadas; todos los reportes obligatorios presentes; p95 de tracking < 500 ms y errores < 1 % a escala completa; usabilidad ≥ 80 %; autorización explícita de Alejandro

17. Gestión de defectos

Cada defecto se registra como Issue con severidad (crítica, alta, media, baja), pasos, evidencia y commit; se enlaza al caso que lo detectó y a la corrección (PR). Un defecto crítico o alto abierto bloquea el GO. La lista consolidada está en el informe final.

18. Métricas

Pruebas ejecutadas y aprobadas por nivel, cobertura de código (unitarias + integración), bugs, vulnerabilidades y code smells de SonarQube, avisos de ZAP, hallazgos de Semgrep y npm/NuGet, p95 y tasa de error de k6, hallazgos de axe y éxito de usabilidad. Valores medidos y su fecha en `informe-final.pdf`.

19. Riesgos residuales

Son los aspectos que no se pudieron cerrar y su tratamiento; la lista vigente está en el informe final (sección de riesgos residuales y bloqueos) y se actualiza en cada candidato.

20. Decisión GO / NO-GO

Se decide con la tabla de criterios de salida (sección 16). Un criterio obligatorio sin evidencia equivale a NO-GO. Lo decide y autoriza Alejandro en Jenkins; sin autorización explícita no hay despliegue final. El resultado vigente está en el informe final.